MC970 - Introdução à programação paralela
Relatório - OpenMP Loops

Daniel de Sousa Cipriano RA: 233228

I. Descrição de sistema

Os exercícios foram executados em um sistema Linux baseado em Ubuntu com a seguinte configuração:

Componente	Descrição
Sistema operacional	Linux Mint 64 bits
Processador	AMD Ryzen 5 5500U com SMT
Núcleos	6 físicos, 12 lógicos
Clock	4.05 GHz
Memória RAM	16 GB DDR4

II. Ferramentas

As principais ferramentas utilizadas para desenvolvimento foram:

Hyperfine: gerador de benchmark. Recebe dois comandos e retorna dados comparativos de desempenho entre eles[1]. Foi utilizada a biblioteca `matplotlib` do Python, em conjunto com benchmarks em formato JSON do Hyperfine, para gerar dados e plotar gráficos de speedup por valores de entrada, eficiência por valores de entrada e tempo de execução serial/paralelo por valores de entrada.

OpenMP: O OpenMP(Open Multi-Processing) é uma API para programação de sistemas multiprocessados com memória compartilhada. A interface fornece diretivas de compilação, bibliotecas de funções e variáveis de ambiente que facilitam e padronizam a implementação de sistemas com multithreading[2].

Gperftools: O Gperftools é um conjunto de ferramentas, distribuído pela Google, para análise de performance em sistemas com multithreading[3]. O software fornece utilitários como o CPU profiler, para perfilamento de código com múltiplas opções de análise[4] e o Heap profiler que fornece análise de heap(alocação dinâmica, vazamentos de memória, etc)[5]. Para esse exercício, foi utilizado o CPU profiler em modo de análise gráfica. Essa configuração gera um grafo orientado de nós, cada nó representa uma função executada e cada aresta representa uma chamada de função, apontando da função origem para a função destino(incluindo funções de kernel). O CPU profiler trabalha coletando amostras periódicas do estado do programa em tempo de execução, registrando em que etapa o fluxo estava em cada medição. A saída gráfica exibe os seguintes campos:

- Nome do binário de execução
- Total samples: total de amostras periódicas coletadas
- Focusing on: amostras efetivamente utilizadas no grafo. Pode ser menos que o total caso sejam aplicados filtros de função, tempo ou ruído
- Dropped nodes with <= x abs(samples): indica descarte de nós que possuem menor amostragem do que o limite mínimo x(pode ser ajustado com - -nodefraction)
- Dropped edges with <= x samples: indica descarte de arestas que possuem menor amostragem do que o limite mínimo x(pode ser ajustado com - -edgefraction)

Dentro dos nós, estão as seguintes informações:

- Nome da função
- Número de amostras de tempo coletadas dentro da função
- Porcentagem do tempo total gasto na função
- Total de amostras cumulativas, incluindo as funções chamadas a partir desta, ou seja, as amostras foram gastas a partir

dessa função, mas não dentro dela diretamente

Por fim, as arestas representam uma chamada à função na direção especificada e junto de cada uma está o número de amostras em que a chamada ocorreu. Em alguns casos foram excluídas funções de kernel, para melhorar a visualização do grafo.

III. OpenMP Parallel For: Matrix Multiplication

A. Objetivo

O objetivo desse exercício foi implementar a versão paralelizada de um programa serial que gera o resultado de uma multiplicação de matrizes. O cálculo da matriz resultante se baseia no algoritmo "ingênuo"de multiplicação de matrizes, que utiliza o método mais tradicional para calcular o produto entre duas matrizes(não utiliza nenhuma otimização). Dadas duas matrizes A e B, o algoritmo utiliza o seguinte cálculo para gerar a matriz resultante C:

$$C_{i,j} = \sum_{k=0}^{m-1} A_{i,k} \cdot B_{k,j} \tag{1}$$

B. Resultados

A versão paralelizada foi implementada com a utilização da diretiva `for` do OpenMP, que automaticamente divide iterações de um laço entre as threads que estão executando no bloco paralelo. A diretiva foi utilizada sem explicitar detalhes de divisão do trabalho entre threads(que poderia ser especificada com `schedule`), garantindo que a própria interface divida igualitariamente da melhor forma a carga de trabalho. Foram testadas entradas para diferentes tamanhos de matrizes quadradas a serem multiplicadas. O método de paralelização foi baseado na divisão de linhas da matriz para as threads, com cada uma calculando os respectivos resultados da multiplicação sobre colunas. Foram utilizadas 4 threads no bloco paralelo. Analisando o gráfico de speedup por dimensão da matriz de entrada(cada valor n no eixo x gera uma matriz n x n) na Figura 1, podemos observar aumento significativo do speedup conforme aumentam os tamanhos de matriz e consequentemente a granularidade do problema, o que reflete a compensação do custo de paralelização conforme o nível de computação aumenta no sistema. Como consequência, um comportamento semelhante pode ser visto no gráfico de eficiência por dimensão de matriz na Figura 2. Também podemos analisar a redução do tempo de execução serial sobre o tempo de execução paralela no gráfico da Figura 3.

Podemos analisar os hot spots do programa, observando o grafo de amostragem de tempo gasto em cada função, para a versão serial e paralela, nas Figuras 4 e 5. Como esperado, a função que consome a maior proporção de tempo do programa é a que realiza a multiplicação de matrizes.

C. Análise com vetorização aplicada

Nesse exercício foi possível aplicar a diretiva `simd` do OpenMP, que garante a vetorização do loop englobado pelo bloco paralelo. A técnica de vetorização utiliza o padrão de arquitetura SIMD(Single Instruction Multiple Data), que aplica simultaneamente uma única instrução da CPU à vários dados do programa. No caso do exercício, a vetorização é aplicada à computação das partes na matriz resultante, realizada paralelamente por cada thread. O gráfico de tempo de execução serial, paralelo e paralelo com aplicação de SIMD pode ser visto na Figura 6. Podemos visualizar o aumento de desempenho da versão paralelizada com o uso de vetorização.

IV. OpenMP Parallel For: Primes

A. Objetivo

O objetivo desse exercício foi implementar a versão paralelizada de um programa serial que calcula o total de números primos menores que um valor limite. Dado um número n natural,

o programa calcula e retorna em iterações de $n \cdot 2$, a quantidade de números primos entre um $n_0 = 1$ e um $n_{max} = 2^n$.

B. Resultados

Novamente, a versão paralelizada foi implementada com a utilização da diretiva `for` do OpenMP, sem especificação de carga de trabalho. O método de paralelização consistiu na divisão do cálculo de primos dentro de um intervalo para cada thread. Foi utilizada a diretiva `reduction` do OpenMP para garantir a soma de cada resultado em variáveis locais de thread, somando a contribuição total na variável original ao fim da execução. Foram utilizadas 4 threads no bloco paralelo. Podemos analisar o gráfico de speedup por valor de entrada no gráfico da Figura 7, vemos que o speedup se mantém em uma faixa que representa o dobro de tempo de execução serial comparada à execução paralela, esse comportamento pode ser explicado pela quantidade fixa de threads utilizadas no bloco paralelo, o que mantém o balanceamento igualitário de carga. Novamente, por consequência do speedup, temos o gráfico de eficiência por valor de entrada repetindo um comportamento semelhante ao anterior na Figura 8. Analisando o gráfico de tempo de execução serial e paralela da Figura 9, temos que o tempo de execução paralela supera o tempo serial para valores crescentes de entrada. Para valores de entrada não tão altos, o tempo de execução das diferentes versões quase se iguala. Novamente, esse comportamento reflete a eficiência da paralelização, quando se lida com problemas de granularidade alta. Analisando os grafos de hot spot nas Figuras 10 e 11, vemos a função que realiza a contagem dos primos ocupando quase todo o tempo de execução do programa.

APÊNDICE

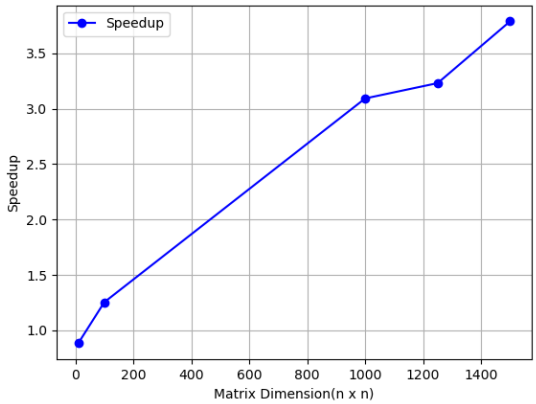


Figura 1: Gráfico de speedup por dimensão da matriz de entrada no exercício OpenMP Parallel For: Matrix Multiplication

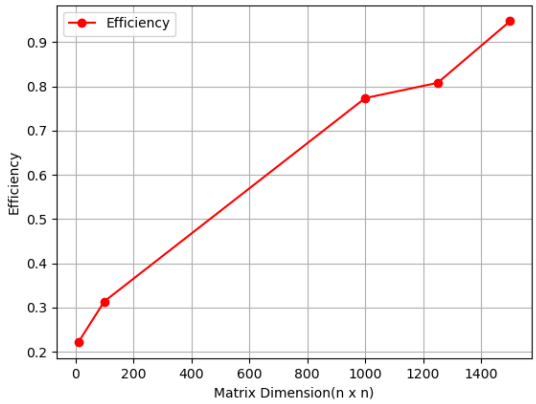


Figura 2: Gráfico de eficiência por dimensão da matriz de entrada no exercício OpenMP Parallel For: Matrix Multiplication

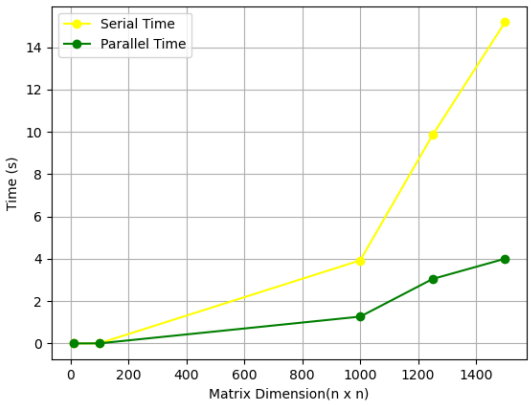


Figura 3: Gráfico de tempo de execução serial e paralela por dimensão da matriz de entrada no exercício OpenMP Parallel For: Matrix Multiplication

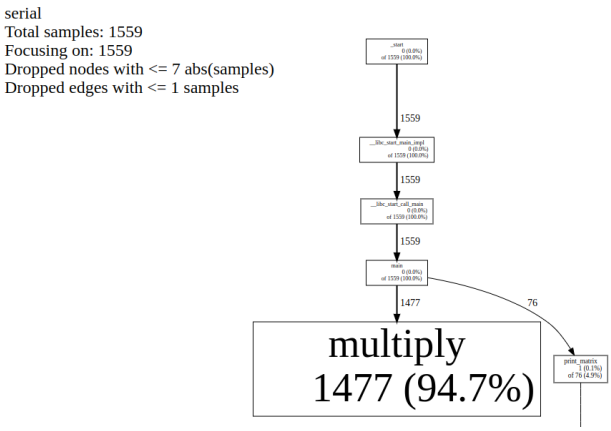


Figura 4: Grafo de amostragem de tempo gasto em cada função, para a versão serial no exercício OpenMP Parallel For: Matrix Multiplication

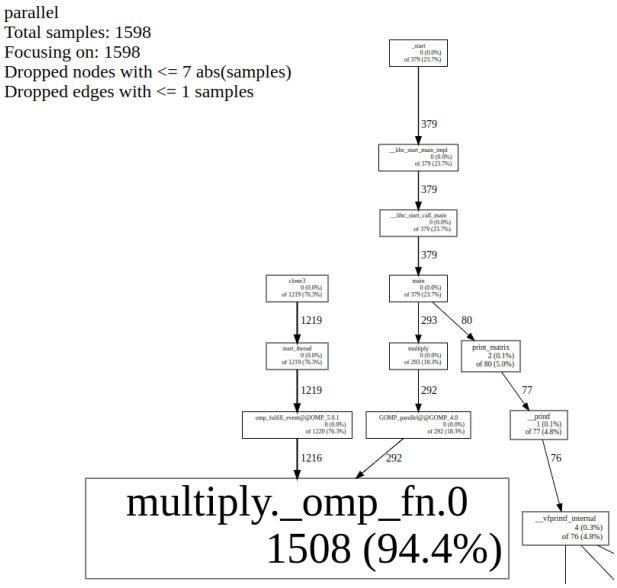


Figura 5: Grafo de amostragem de tempo gasto em cada função, para a versão paralelizada no exercício OpenMP Parallel For: Matrix Multiplication

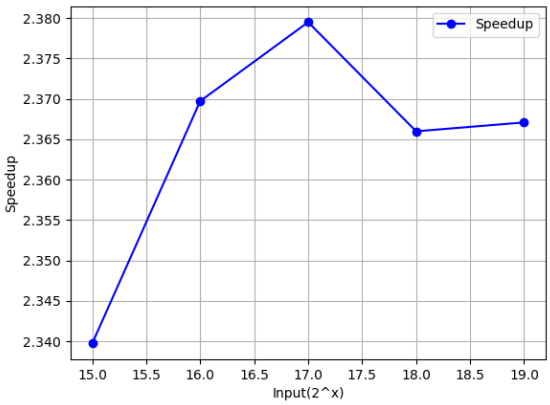


Figura 7: Gráfico de speedup por valor de entrada no exercício OpenMP Parallel For: Primes

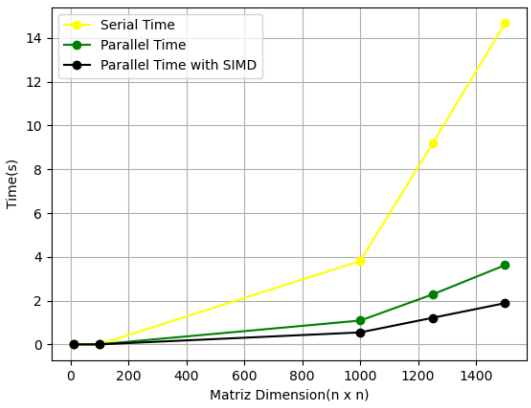


Figura 6: Gráfico de tempo de execução serial, paralelo e paralelo com SIMD no exercício OpenMP Parallel For: Matrix Multiplication

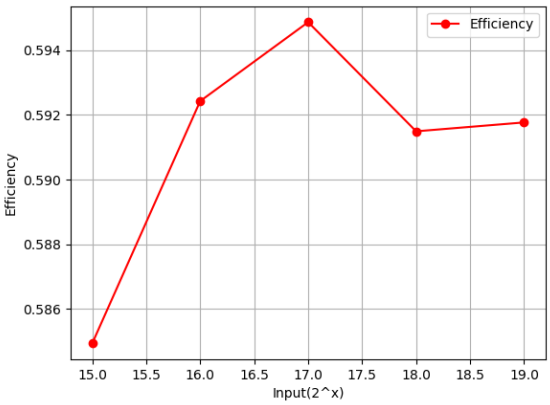


Figura 8: Gráfico de eficiência por valor de entrada no exercício OpenMP Parallel For: Primes

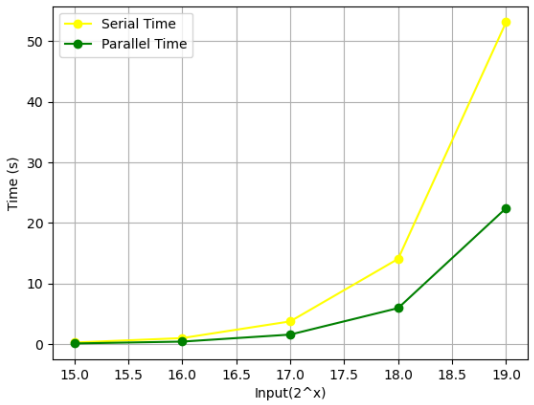


Figura 9: Gráfico de tempo de execução serial e paralela por valor de entrada no exercício OpenMP Parallel For: Primes

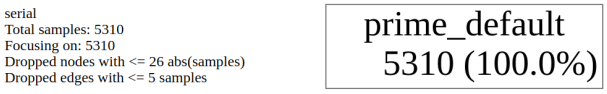


Figura 10: Grafo de amostragem de tempo gasto em cada função, para a versão serial no exercício OpenMP Parallel For: Primes

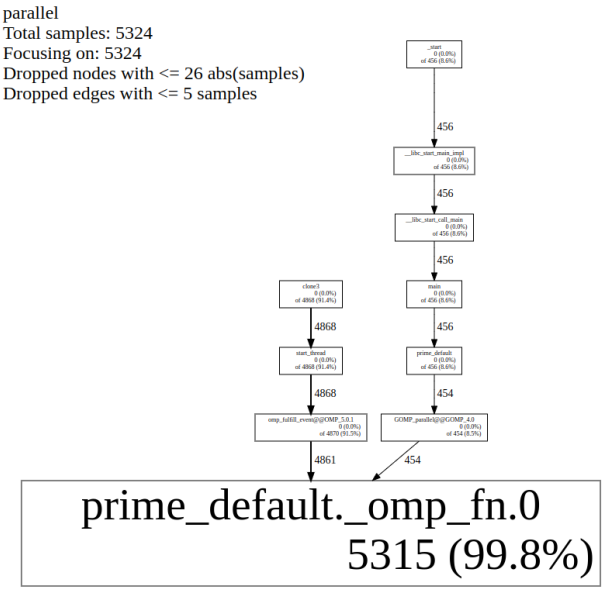


Figura 11: Grafo de amostragem de tempo gasto em cada função, para a versão paralelizada no exercício OpenMP Parallel For: Primes

Referências

[1] PETER, D. hyperfine. Disponível em: <https://github.com/sharkdp/hyperfine>.

[2] TIM.LEWIS. About Us. Disponível em: <https://www.openmp.org/about/about-us/>.

[3] gperftools/gperftools. Disponível em: <https://github.com/gperftools/gperftools>.

[4] RIDGERUN DEVELOPER WIKI. GPerfTools — Profiling with GPerfTools — RidgeRun. Disponível em: <https://developer.ridgerun.com/wiki/index.php/Profiling_with_GPerfTools⟩.Acessoem:19abr.2025>.

[5] AUTHOR :, O. Gperftools Heap Profiler. Disponível em: <https://gperftools.github.io/gperftools/heapprofile.html>. Acesso em: 19 abr. 2025.